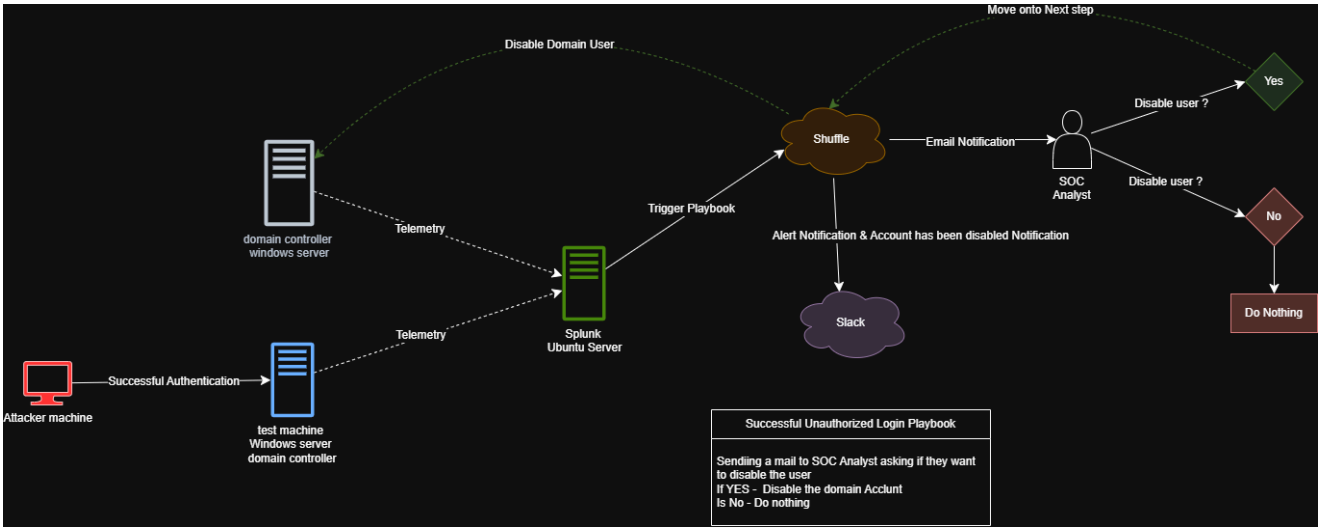
# SOAR-Based Automated Incident Response for RDP Activity in Active Directory

Overview of HomeLab :




This home lab simulates a real-world Security Operations Center (SOC) environment by integrating identity management, log monitoring, threat detection, and automated incident response. The primary objective of this setup is to monitor Remote Desktop Protocol (RDP) login activity in domain user accounts and respond to suspicious events by disabling compromised accounts with analyst approval.

The lab begins with the configuration of a Windows Server acting as a Domain Controller, where multiple domain user accounts are created using Active Directory. A Windows client machine is joined to this domain to simulate user activity such as authentication and RDP logins. These activities generate security logs that are crucial for monitoring and detection.

To enable centralized log collection and analysis, a SIEM solution is deployed using Splunk on an Ubuntu machine. A Universal Forwarder is installed on the Windows client system to continuously send Windows Security Event Logs to the Splunk server. A dedicated index is created in Splunk to store and manage these logs efficiently.

Detection is implemented by configuring alert rules in Splunk that monitor specific event IDs related to RDP login activity. Whenever a login occurs, Splunk generates an alert containing important details such as username, source IP address, and timestamp. This alert serves as the starting point for the automated response workflow.

For automation, the lab utilizes Tines to build a playbook that processes alerts and initiates response actions. The alert from Splunk is sent to Tines through a webhook, triggering the workflow. An HTTP request action is then used to send real-time notifications to Slack and email, ensuring that the SOC analyst is immediately informed about the detected activity.

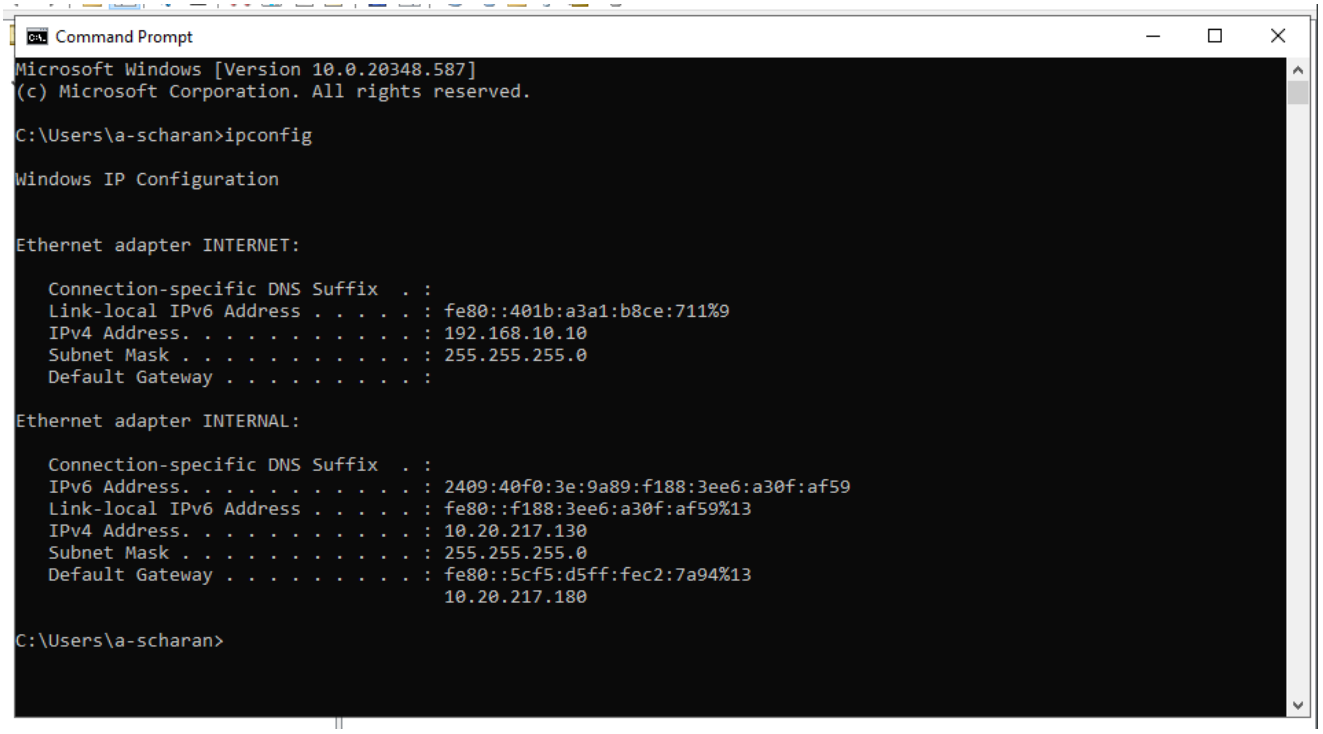
A key feature of this workflow is the inclusion of a human-in-the-loop decision-making process. Through a Page Action interface in Tines, the analyst is presented with an option to either allow or disable the user account. This step ensures that automated responses are validated before execution, reducing the risk of false positives.

If the analyst chooses to proceed with disabling the account, a condition check evaluates the input and triggers the next step in the workflow. An HTTP POST request is sent to a custom-built Flask API running on the Windows Server. This API acts as a bridge between the SOAR platform and Active Directory.

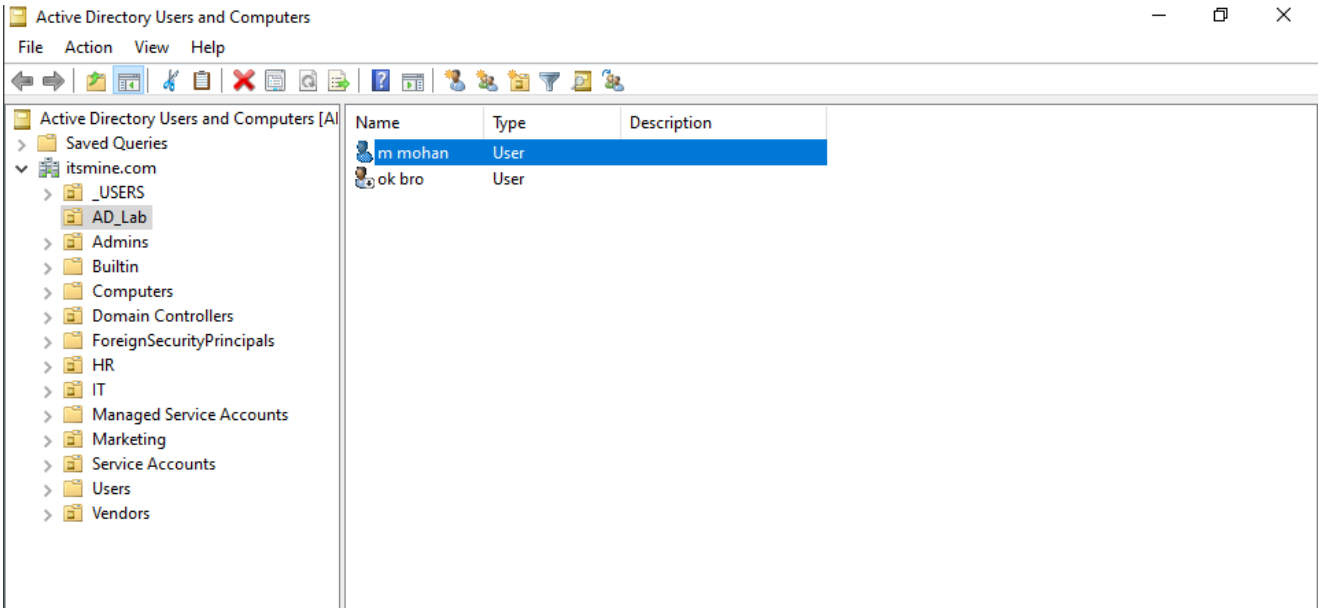
The Flask application executes a PowerShell command that disables the specified user account in Active Directory. This demonstrates how automation tools can interact with system-level administrative functions securely and efficiently.

Once the action is completed, a confirmation message is sent back to Slack, notifying the SOC team that the user account has been successfully disabled. The final validation is performed by checking the Active Directory console, where the disabled account is visually indicated.

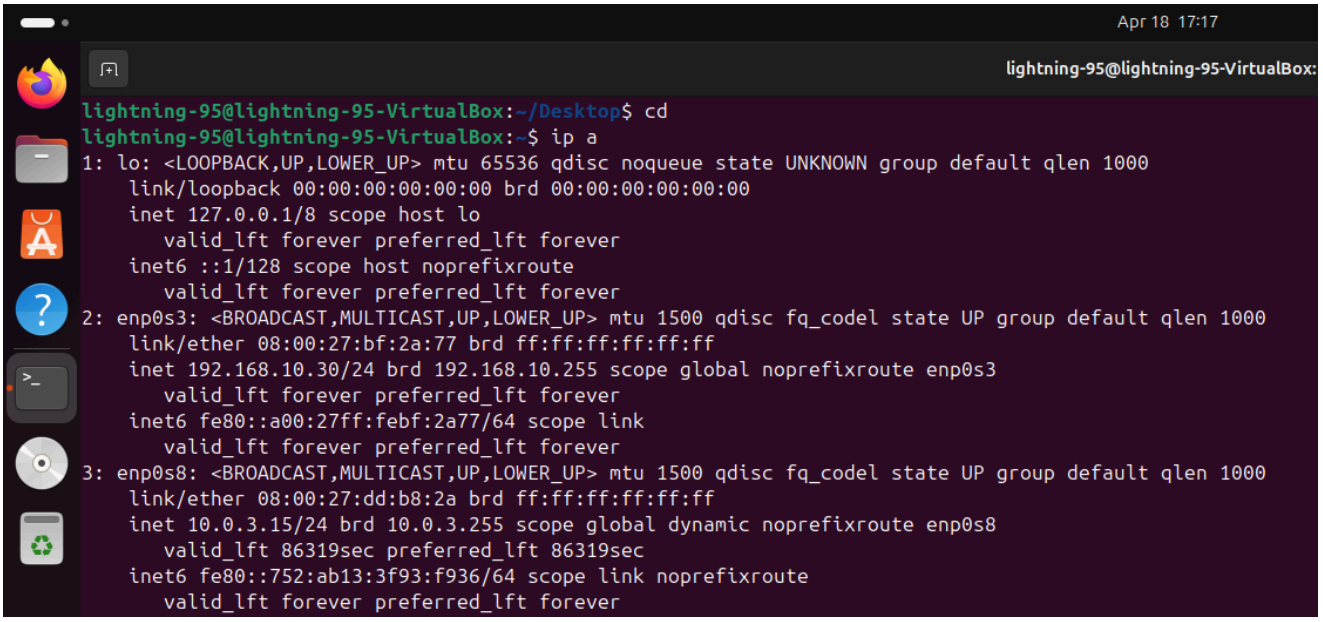
Overall, this home lab successfully demonstrates an end-to-end incident response pipeline, covering detection, alerting, analyst decision-making, automated response, and final validation. It reflects real-world SOC operations by combining SIEM and SOAR technologies with identity management systems.




- This screenshot shows the network configuration of the Windows Server using `ipconfig`, displaying multiple adapters. These adapters are used for internal domain communication and external internet connectivity.
- It confirms that the server is properly configured with IP addresses, subnet masks, and gateways.



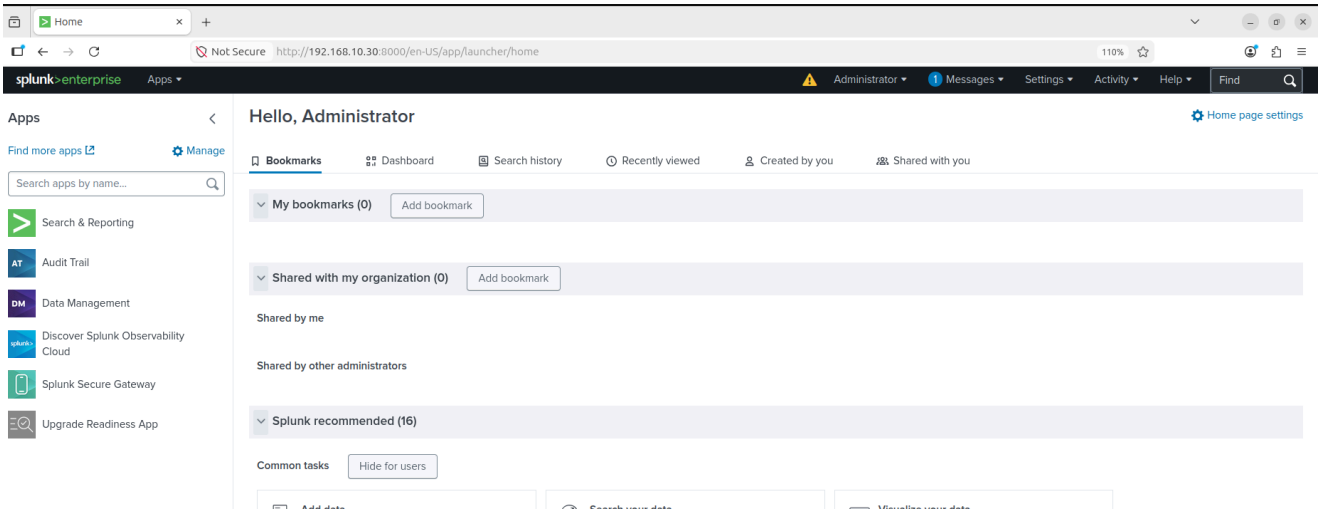

- This screenshot shows the Active Directory Users and Computers console with sample domain users created for testing. These users are used to simulate login activities and security events.




- This screenshot shows the Ubuntu machine's network configuration using `ip a`, confirming connectivity and assigned IP addresses.

```
lightning-95@lightning-95-VirtualBox: /opt/splunk/bin
lightning-95@lightning-95-VirtualBox:/opt/splunk/bin$ sudo ./splunk start --run-as-root

Splunk> Now with more code!

Checking prerequisites...
        Checking http port [8000]: open
        Checking mgmt port [8089]: open
        Checking appserver port [127.0.0.1:8065]: open
        Checking kvstore port [8191]: open
        Checking configuration... Done.
New certs have been generated in '/opt/splunk/etc/auth'.
        Checking critical directories...        Done
        Checking indexes...
                Validated: _audit _configtracker _dm_summary _dsappevent _dsclient _dsphonehome _internal _introspection _
_lab main summary
        Done
        Checking filesystem compatibility...  Done
        Checking conf files for problems...
        Done
        Checking default conf files for edits...
        Validating installed files against hashes from '/opt/splunk/splunk-10.2.2-80b90d638de6-linux-amd64-manifest'
        All installed files intact.
        Done
All preliminary checks passed.

Starting splunk server daemon (splunkd)...
Done


Waiting for web server at http://127.0.0.1:8000 to be available..........................  Done


If you get stuck, we're here to help.
Look for answers here: http://docs.splunk.com

The Splunk web interface is at http://lightning-95-VirtualBox:8000
```

- This screenshot shows Splunk being started using command-line interface, with logs confirming service initialization.




- the Splunk web interface dashboard, confirming that the service is accessible through a browser.
- It provides access to apps, search, and monitoring features.

splunk>enterprise Apps

# Indexes

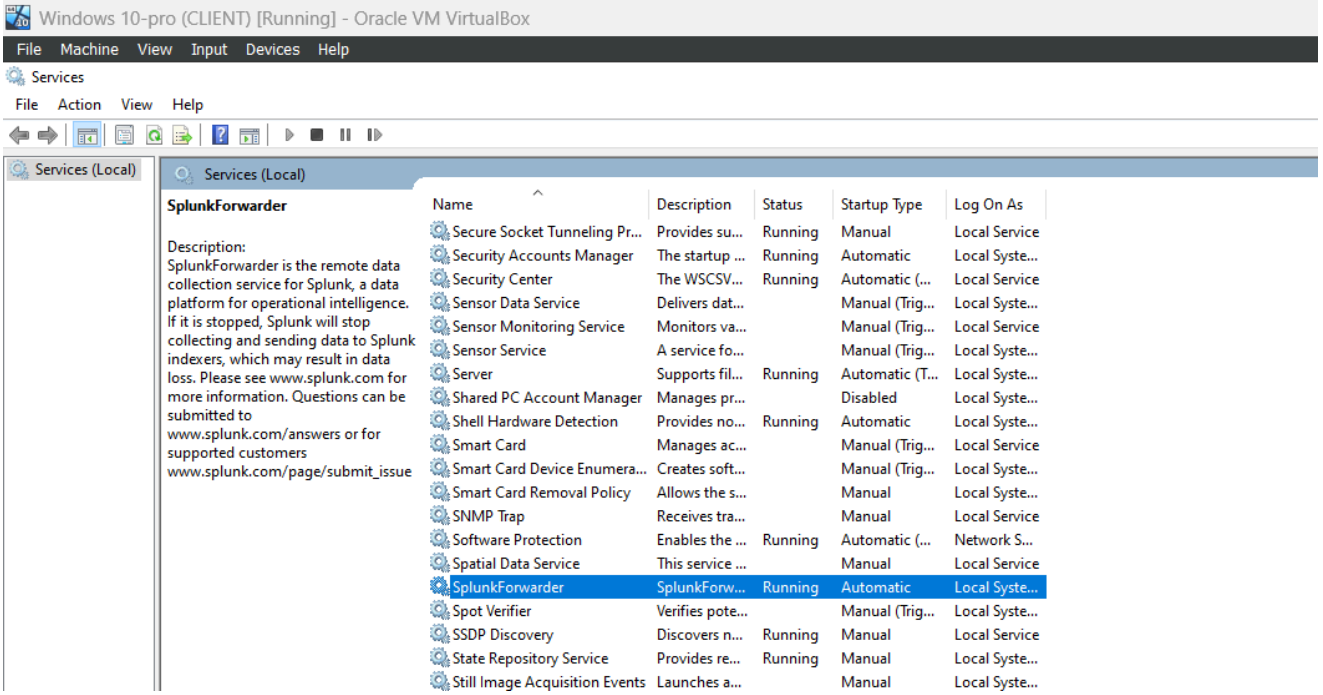
A repository for data in Splunk Enterprise. Indexes reside in flat files on the Splunk Enterprise

Showing 1-17 of 17 Indexes

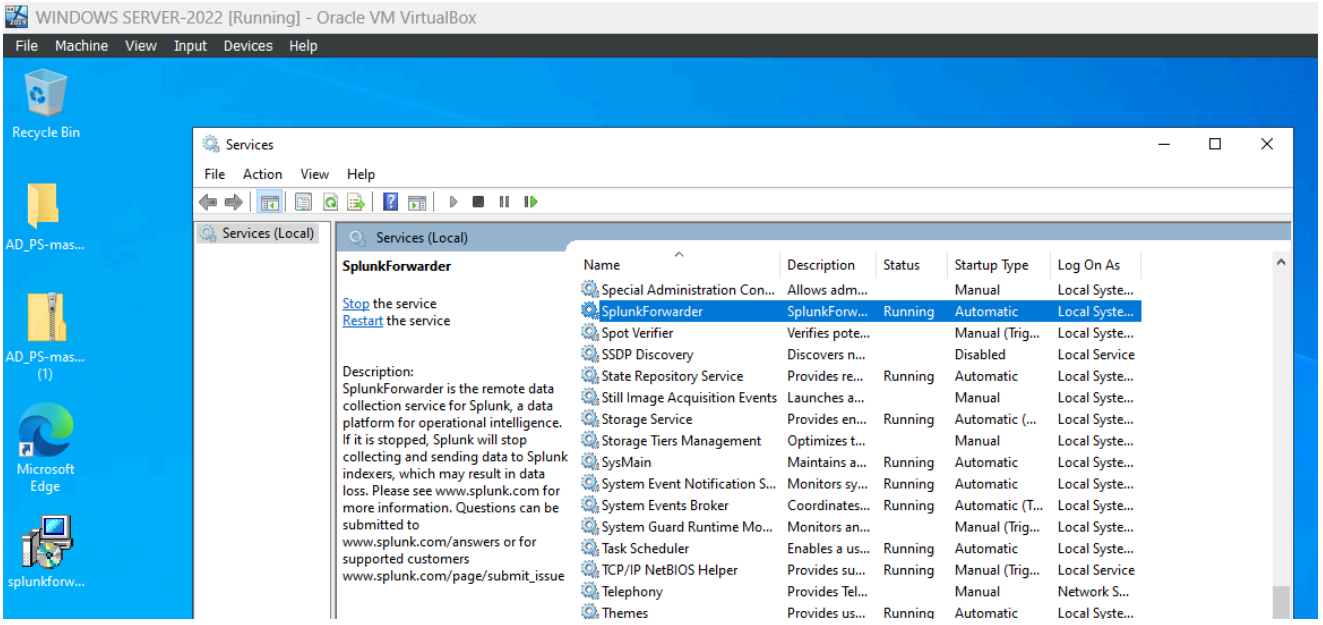
Search...

| Name | Status | Type | App |
|---|---|---|---|
| _audit | Active | Events | system |
| _configtracker | Active | Events | system |
| _dm_summary | Active | Metrics | system |
| _dsappevent | Active | Events | SplunkDeploymentServerConfig |
| _dsclient | Active | Events | SplunkDeploymentServerConfig |
| _dsphonehome | Active | Events | SplunkDeploymentServerConfig |
| _internal | Active | Events | system |
| _introspection | Active | Events | system |
| _metrics | Active | Metrics | system |
| _metrics_rollup | Active | Metrics | system |
| _telemetry | Active | Events | system |
| _thefishbucket | Active | Events | system |
| history | Active | Events | system |
| home_lab | Active | Events | search |
| main | Active | Events | system |
| splunklogger | Inactive | Events | system |
| summary | Active | Events | system |

- showing the list of indexes in Splunk, including the custom index **home_lab** created for this project.
- This index stores logs specifically from the domain client machines.

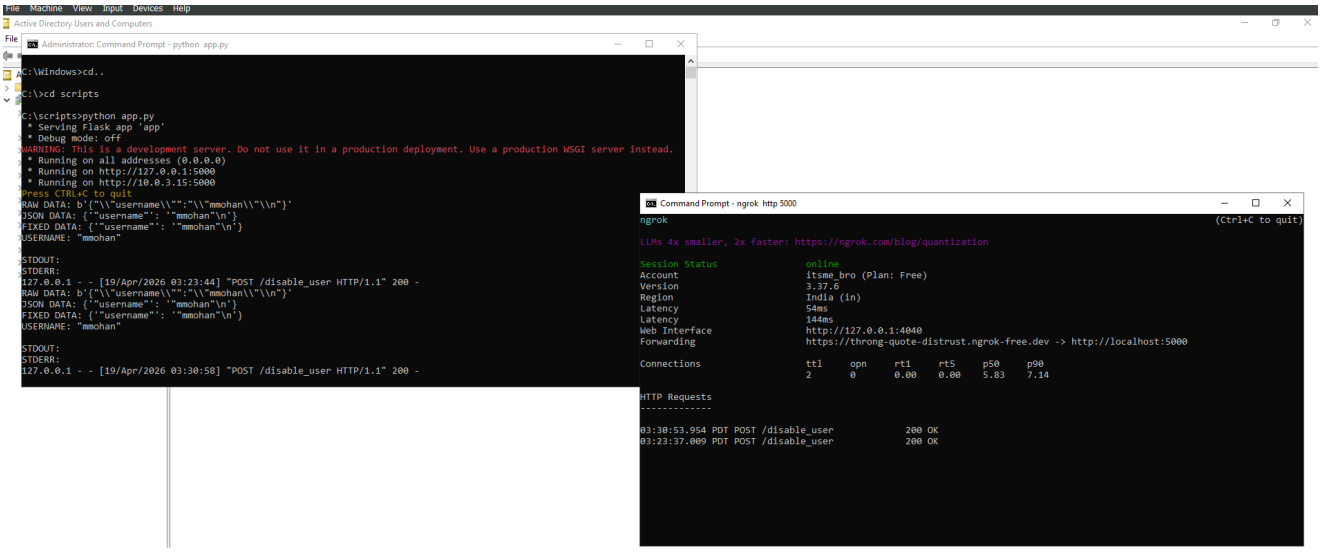



- he Splunk Forwarder service running on the Windows client machine.
- It confirms that logs are actively being forwarded to the Splunk server.

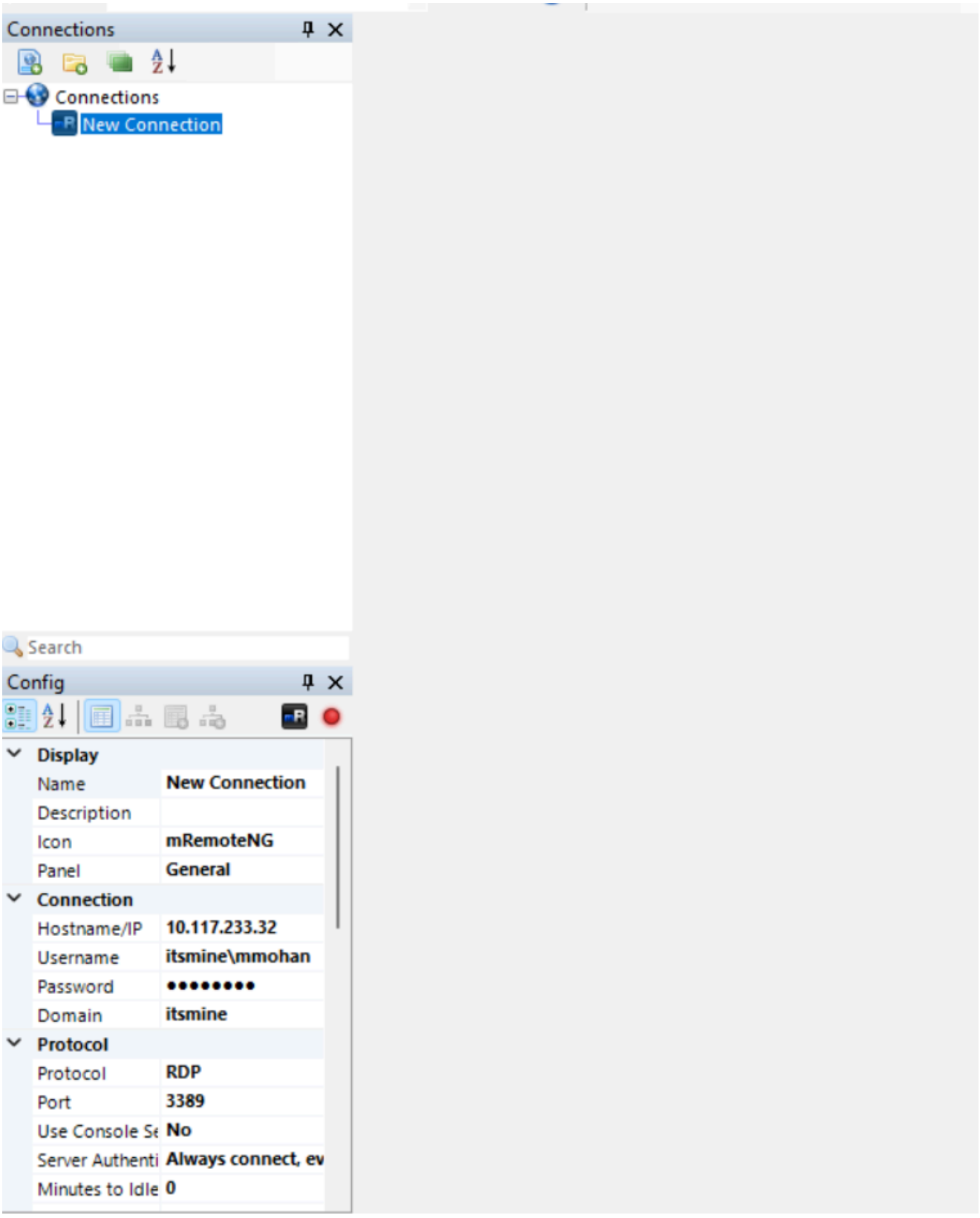



- showing services running on the Windows Server, including system and security services.
- It confirms that the server is operational and ready for domain operations.

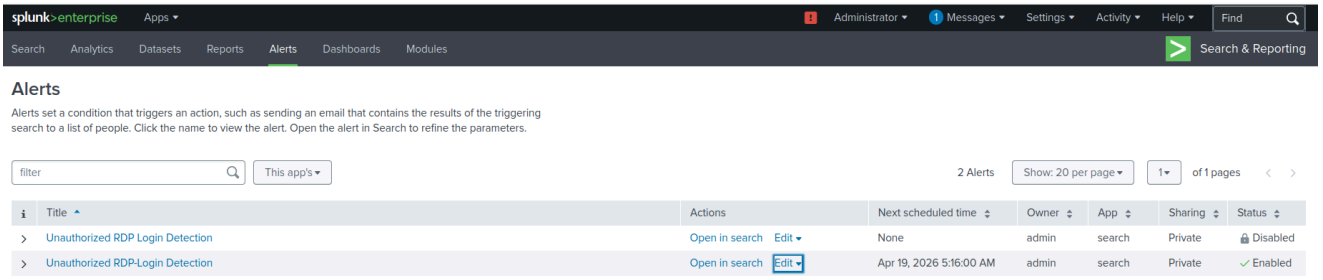
and this is the server i'm running on windows server to perform the disable action from the SOAR playbook , through a POST request.

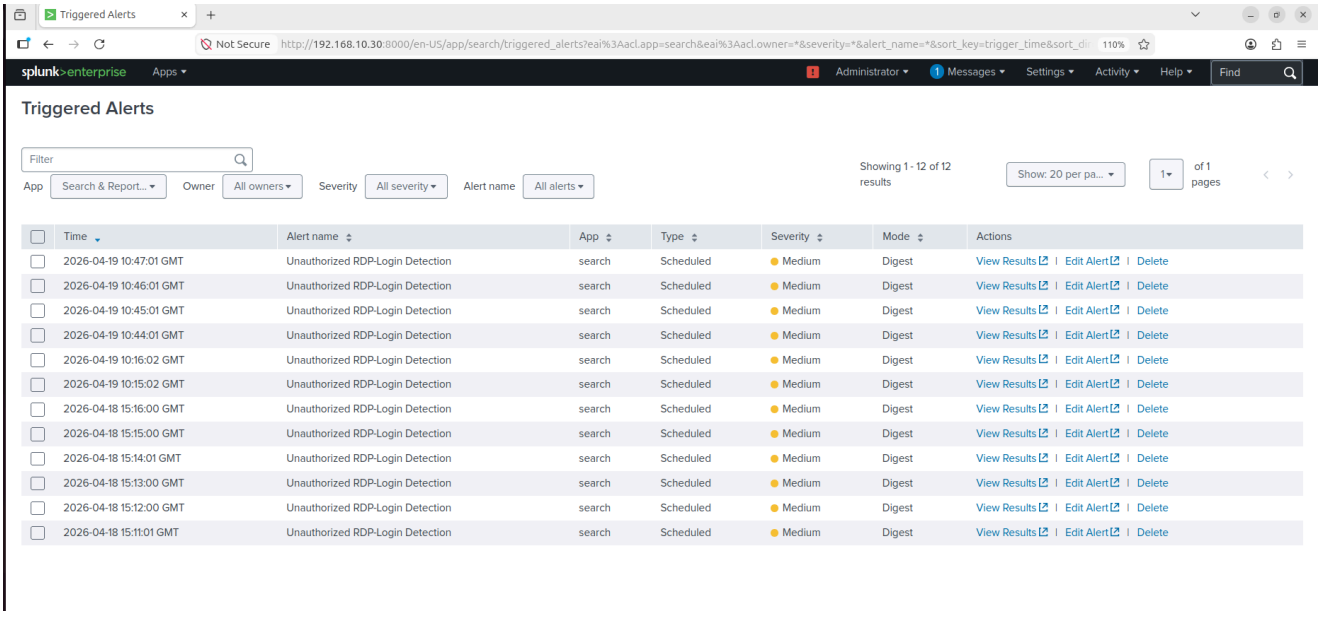



- the Flask application running locally on port 5000.
- It indicates that the API server is active and ready to receive requests.
- ngrok running and creating a secure public URL for the local Flask server.
- This allows external systems like Tines to communicate with the API.

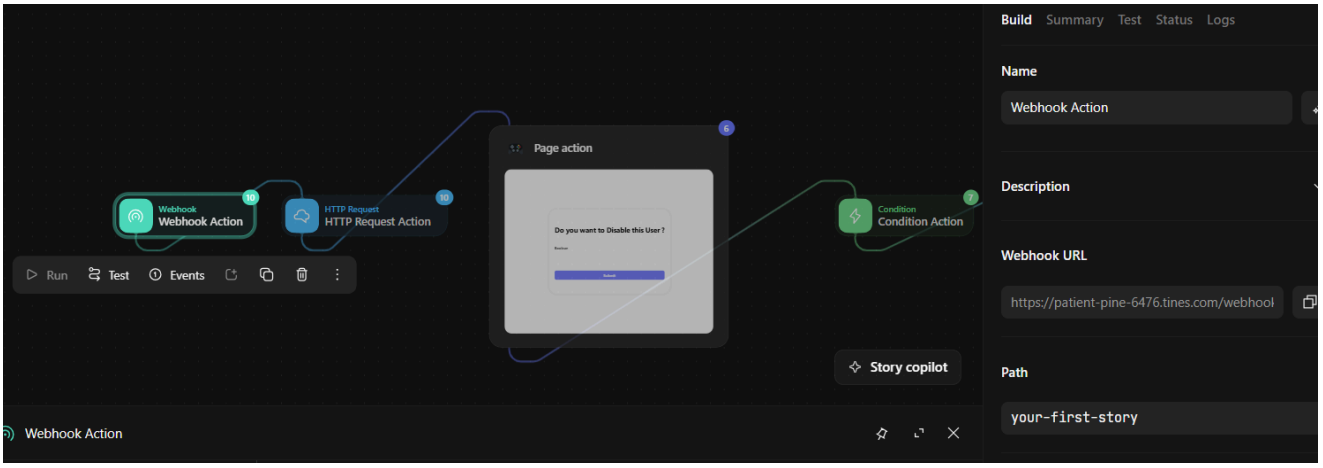



- the RDP connection setup in mRemoteNG with IP, username, and domain details.
- It is used to simulate login activity to trigger alerts.

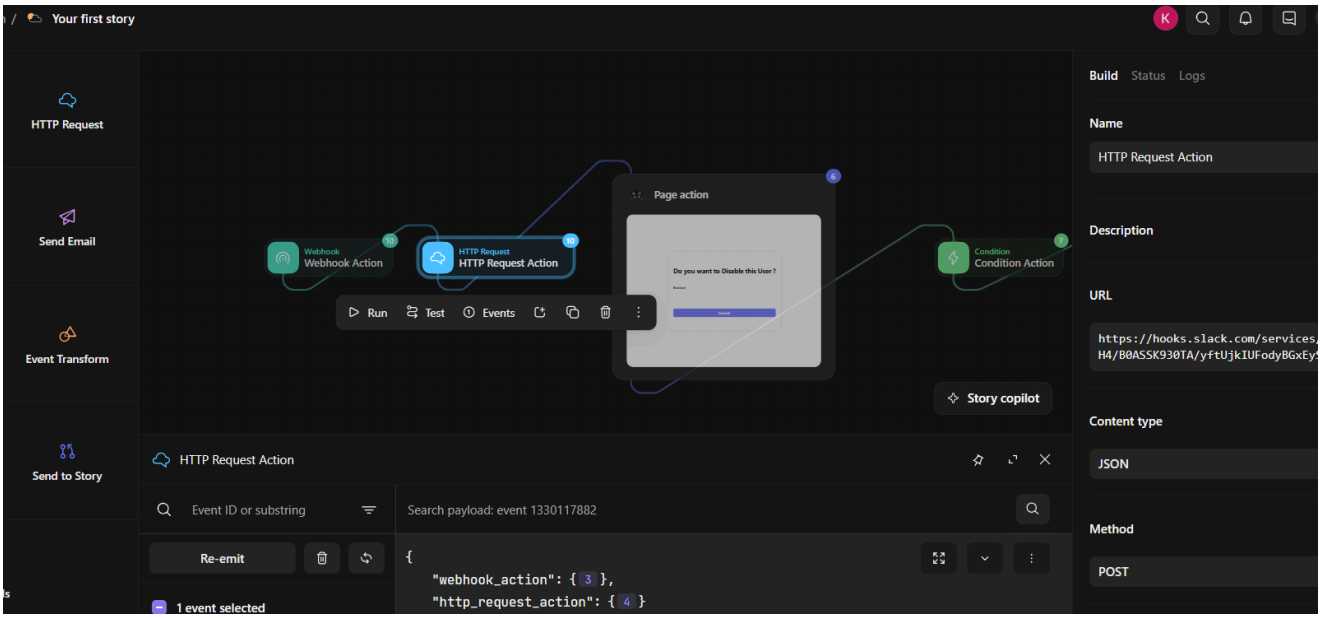


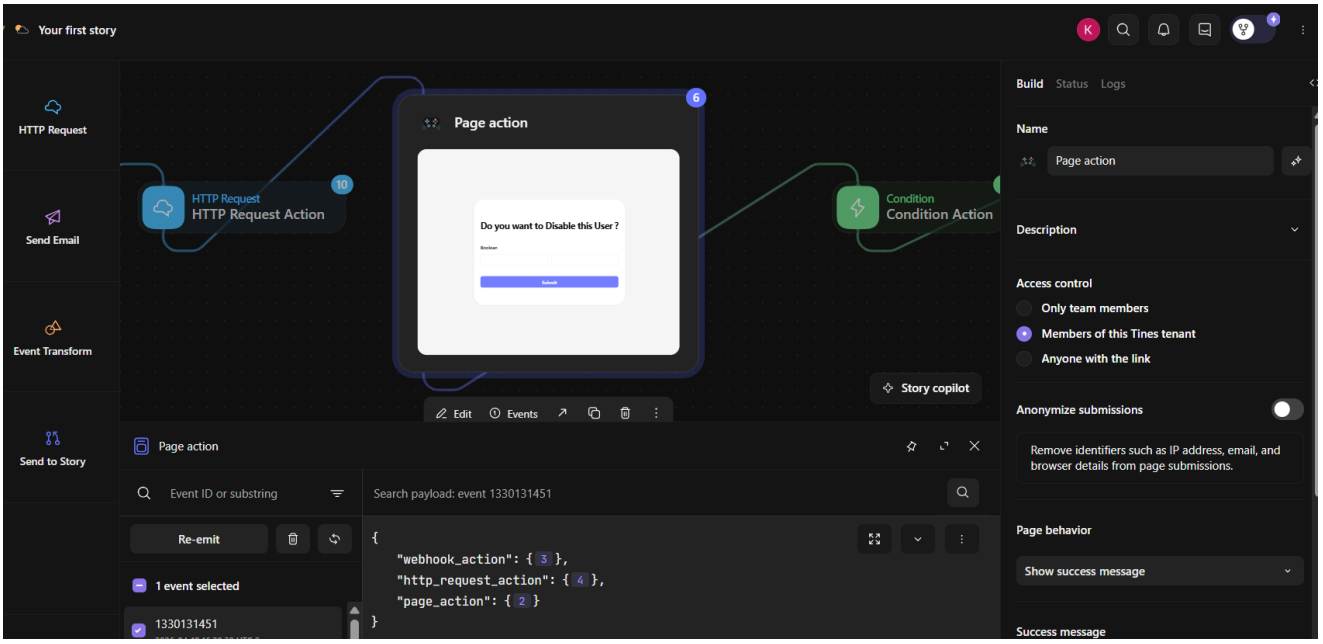

- hows alerts generated in Splunk based on RDP login activity.
- It confirms that detection rules are working correctly.



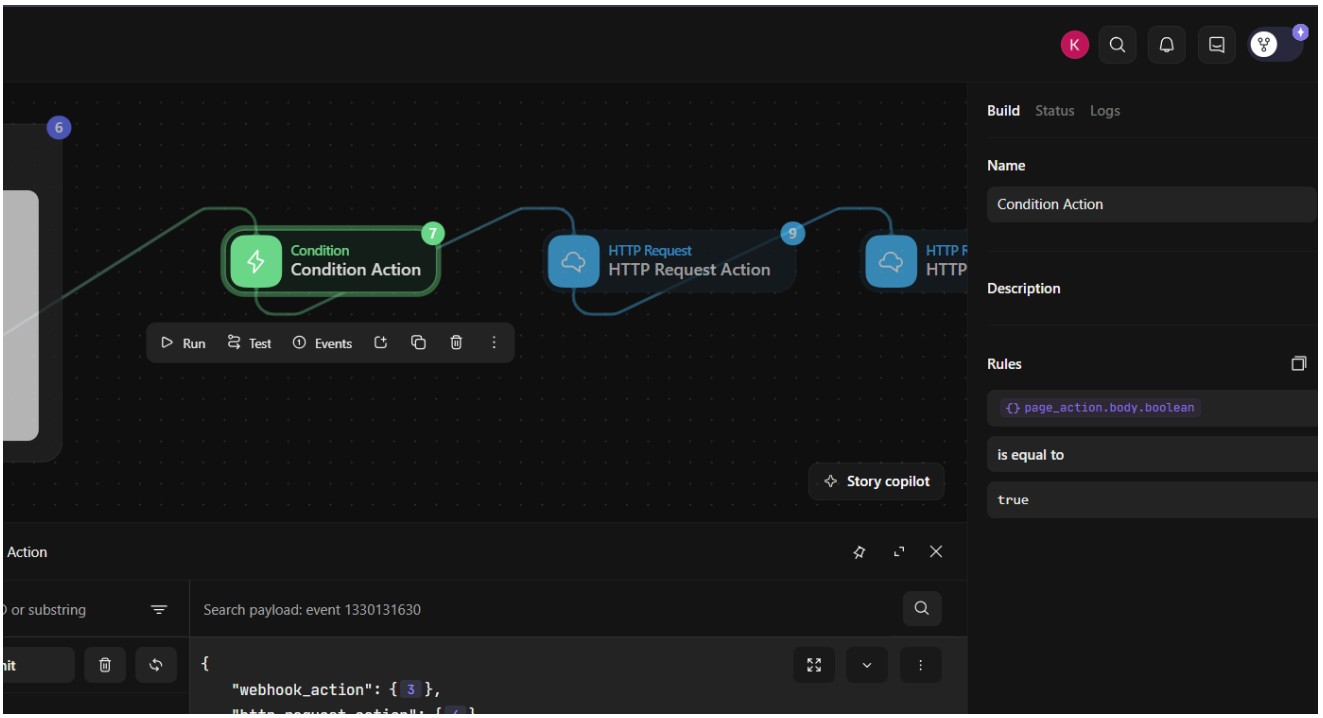

- this webhook action in Tines used to receive alerts from Splunk.
- It acts as the entry point for the automation workflow.



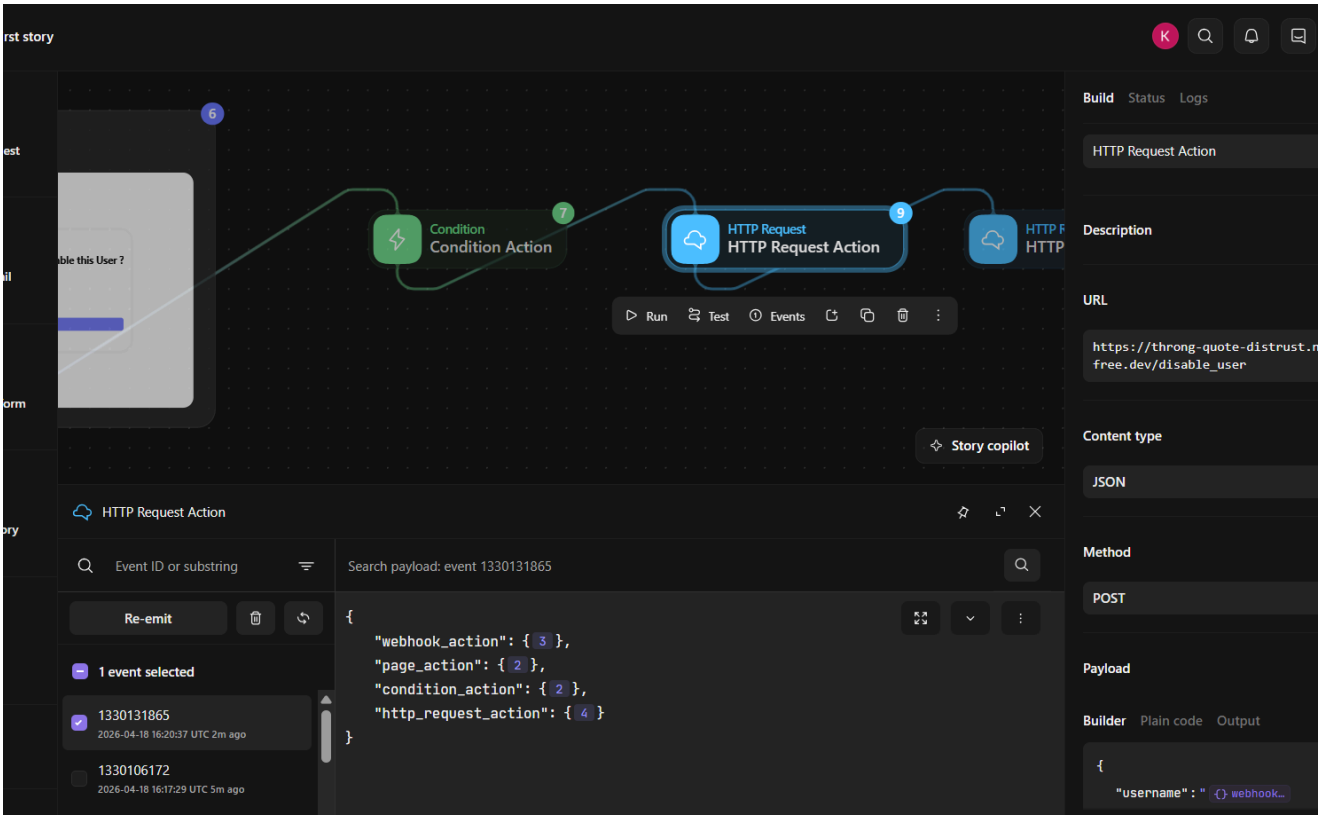

- the HTTP request action used to send alert messages to Slack.
- It includes details like alert name, username, and source IP.



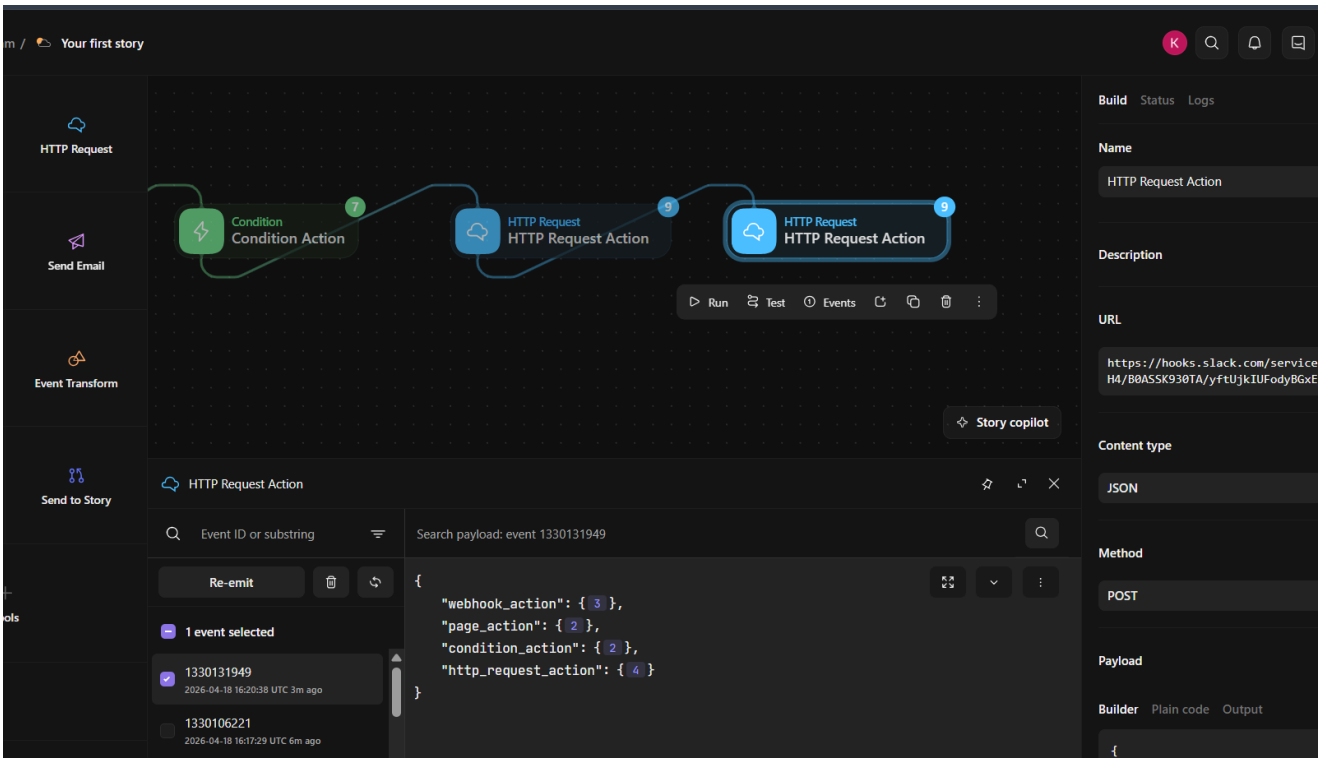

- This screenshot shows the user interface where the analyst selects whether to disable the account.
- It provides a simple Yes/No decision option.
- This step ensures human validation before taking critical actions. It prevents accidental account disabling due to false positives.



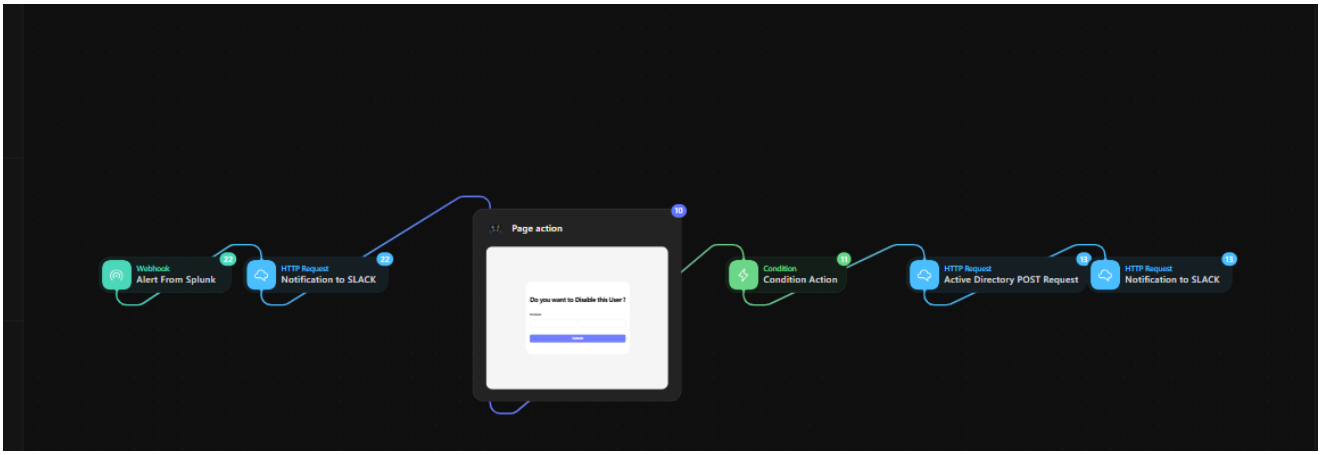

- the condition action that checks the analyst's input.
- It determines whether the workflow should proceed.



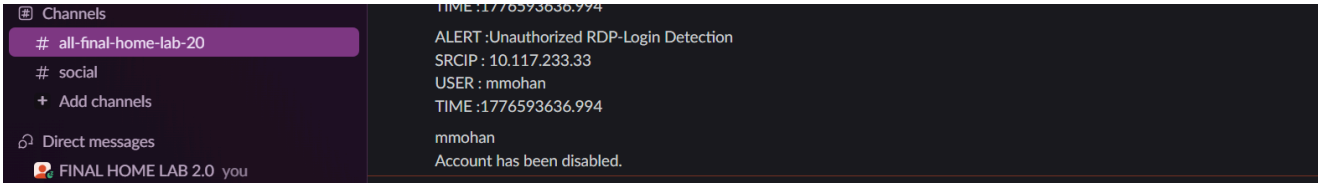

- the HTTP request sending a POST request to the Flask server.
- this triggers the user disable action in Active Directory.



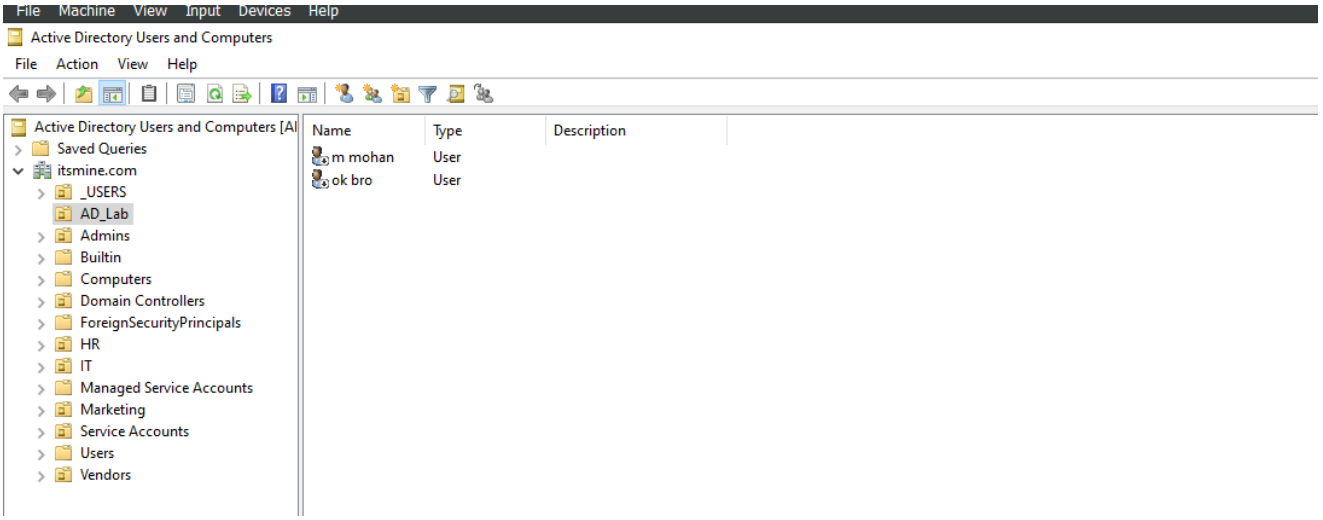

- This screenshot shows the confirmation message sent to Slack after disabling the user.
- It informs the SOC team about the completed action.




- Overall Playbook




- the alert message sent to Slack with details like username, source IP, and timestamp.
- It provides context about the detected activity.
- message confirming that the user account has been disabled.
- It verifies that the response action was successful.




- This screenshot shows the Active Directory console with the user account disabled (down arrow icon).
- It confirms the final outcome of the automation workflow.